

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME:** Design and Analysis of Algorithms

**COURSE CODE:** CSA06

### **COURSE OUTCOME COVERED**

**CO3:** Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication (BL4, BL5)

**Assessment Tool Weightage:** 40%

**QUESTIONS:** 10

**Total Marks:** 50

Annexure A

### **Experiments**

#### ***Code of Conduct:***

*I Thanveer T 192424107 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** Thanveer T

**Register Number:** 192424107

## Experiments — Solutions

Name: Thanveer T

Register Number: 192424107

### Experiment 1: Performance Analysis of Merge Sort (Best, Average, Worst Case)

***Aim:***

To analyze the performance of Merge Sort across different input sizes by measuring execution time for best, average, and worst-case scenarios, and to interpret how the divide-and-conquer strategy ensures consistent performance.

***Algorithm Design & Approach:***

Merge Sort recursively divides the input array into two halves until sub-arrays of size one remain, then merges the sorted halves back together. Because merging two sorted sub-arrays of total size  $n$  always costs  $O(n)$  regardless of the initial arrangement of elements, Merge Sort's performance does not depend on the original ordering of the input — only on its size. Datasets of increasing sizes (e.g. 1000, 5000, 10000, 50000 elements) are generated in three configurations — already sorted (best/typical), randomly shuffled (average), and reverse sorted (worst) — and Merge Sort is timed on each.

***Pseudocode:***

```
MERGE_SORT(A, low, high)
    if low < high
        mid = (low + high) / 2
        MERGE_SORT(A, low, mid)
        MERGE_SORT(A, mid+1, high)
        MERGE(A, low, mid, high)

MERGE(A, low, mid, high)
    create temp arrays L[low..mid], R[mid+1..high]
    i = low ; j = mid+1 ; k = low
    while i <= mid and j <= high
        if A[i] <= A[j]: A_temp[k++] = A[i++]
        else:           A_temp[k++] = A[j++]
    copy remaining elements of L and R into A_temp
    copy A_temp back into A[low..high]
```

***Time / Space Complexity Analysis:***

At every recursion level the array is split into two halves ( $\log n$  levels), and merging all sub-arrays at a given level costs  $O(n)$  in total, giving an overall time complexity of  $O(n \log n)$  for the best, average, and worst cases alike — Merge Sort has no separate worst-case blow-up, unlike Quick Sort. Space complexity is  $O(n)$  due to the auxiliary arrays used during merging.

***Expected Observations & Graph Interpretation:***

Recorded execution times for sorted, random, and reverse-sorted inputs of the same size are expected to be nearly identical (differing only by small constant-factor noise), while plotting execution time against  $n$  on a log-log or  $n$ -log( $n$ ) scaled axis produces a straight line, confirming the  $O(n \log n)$  growth trend across all three cases.

***Conclusion & Justification:***

The experiment confirms that the divide-and-conquer structure of Merge Sort — splitting the problem independent of data values and merging with a fixed-cost linear pass — guarantees  $O(n \log n)$  performance

regardless of input order, making it a reliable, predictable choice when consistent worst-case behaviour is required (e.g. real-time or safety-critical systems).

## Experiment 2: Effectiveness of Pivot Selection Strategies in Quick Sort

### *Aim:*

To evaluate first-element, last-element, and random pivot selection strategies in Quick Sort by comparing their execution times across different input conditions and to justify which strategy best avoids the worst case.

### *Algorithm Design & Approach:*

The same Quick Sort partitioning logic is implemented three times, differing only in how the pivot is chosen before partitioning: (1) always the first element, (2) always the last element, (3) a uniformly random element swapped into the last position before partitioning. Each variant is executed on random, sorted, and reverse-sorted datasets of equal size, and execution time is recorded for every combination.

### *Pseudocode:*

```
QUICK_SORT(A, low, high)
    if low < high
        p = PARTITION(A, low, high)    // pivot chosen per strategy
        QUICK_SORT(A, low, p-1)
        QUICK_SORT(A, p+1, high)

PARTITION(A, low, high)    // Lomuto scheme, pivot = A[high]
    pivot = A[high]
    i = low - 1
    for j = low to high-1
        if A[j] <= pivot
            i = i + 1
            swap A[i], A[j]
    swap A[i+1], A[high]
    return i + 1

// Pivot-selection variants (choose before calling PARTITION):
//   First pivot:  swap A[low], A[high]  before partitioning
//   Random pivot: r = random(low, high); swap A[r], A[high]
```

### *Time / Space Complexity Analysis:*

Average-case time complexity for all three strategies is  $O(n \log n)$  on random data. However, first-pivot and last-pivot strategies degrade to  $O(n^2)$  on already-sorted or reverse-sorted input, because every partition splits the array into a size-0 and a size-( $n-1$ ) piece. Random pivot selection makes this pathological split extremely unlikely regardless of input order, so its expected time complexity remains  $O(n \log n)$  even on sorted/reverse-sorted data — the worst case still exists but is no longer tied to predictable input patterns.

### *Expected Observations & Graph Interpretation:*

First- and last-pivot strategies are expected to show a sharp execution-time spike on sorted and reverse-sorted inputs compared to random inputs, while the random-pivot strategy shows comparatively flat execution time across all three input conditions, closely tracking the  $O(n \log n)$  curve.

### *Conclusion & Justification:*

Random pivot selection provides the most robust and optimal performance because it removes the dependency between input arrangement and worst-case behaviour; it is therefore the recommended strategy in practice, often combined with median-of-three selection for further stability.

## Experiment 3: Binary Search vs Linear Search — Comparative Analysis

***Aim:***

To compare the performance of Binary Search and Linear Search on datasets of varying sizes and to justify the suitability of each method based on input conditions and complexity.

***Algorithm Design & Approach:***

A sorted dataset of increasing sizes is prepared. Linear Search scans sequentially from the start until the target is found or the array ends. Binary Search repeatedly halves the search interval using the sorted property of the data. Both algorithms are timed while searching for elements at varying positions (start, middle, end, absent) across all dataset sizes.

***Pseudocode:***

```
LINEAR_SEARCH(A, n, key)
  for i = 0 to n-1
    if A[i] == key: return i
  return -1

ITERATIVE_BINARY_SEARCH(A, n, key)
  low = 0 ; high = n - 1
  while low <= high
    mid = low + (high - low) / 2
    if A[mid] == key: return mid
    else if A[mid] < key: low = mid + 1
    else: high = mid - 1
  return -1
```

***Time / Space Complexity Analysis:***

Linear Search:  $O(1)$  best case,  $O(n)$  average and worst case. Binary Search:  $O(1)$  best case,  $O(\log n)$  average and worst case, but requires the data to already be sorted (an  $O(n \log n)$  precondition if sorting is needed first).

***Expected Observations & Graph Interpretation:***

As dataset size grows, Linear Search's execution time increases linearly, while Binary Search's execution time grows only logarithmically — for  $n = 1,000,000$ , Linear Search may need up to a million comparisons in the worst case, while Binary Search needs at most about 20.

***Conclusion & Justification:***

Binary Search is dramatically more efficient for large, static, sorted datasets, while Linear Search remains preferable for small datasets, unsorted data, or data that changes so frequently that maintaining sorted order would outweigh the search-time benefit.

**Experiment 4: Strassen's Matrix Multiplication vs Standard Matrix Multiplication*****Aim:***

To evaluate the performance difference between Strassen's algorithm and the standard triple-nested-loop matrix multiplication method, and to justify scenarios where Strassen's method offers a computational advantage.

***Algorithm Design & Approach:***

Both algorithms are implemented and executed on square matrices of increasing size (e.g.  $64 \times 64$ ,  $128 \times 128$ ,  $256 \times 256$ ,  $512 \times 512$ ), padding sizes to the next power of two where required by Strassen's divide-and-conquer recursion. Execution time is recorded for each matrix size and method.

***Pseudocode:***

```
STANDARD_MULTIPLY(A, B, n)    // n x n matrices
  for i = 0 to n-1
```

```

    for j = 0 to n-1
        C[i][j] = 0
        for k = 0 to n-1
            C[i][j] += A[i][k] * B[k][j]
    return C

STRASSEN_MULTIPLY(A, B, n)    // n x n, n a power of 2
    if n == 1: return [[ A[0][0] * B[0][0] ]]
    split A, B into four (n/2)x(n/2) submatrices:
        A11,A12,A21,A22 and B11,B12,B21,B22
    M1 = STRASSEN(A11+A22, B11+B22)
    M2 = STRASSEN(A21+A22, B11)
    M3 = STRASSEN(A11, B12-B22)
    M4 = STRASSEN(A22, B21-A11)    // note: A22 * (B21 - B11)
    M5 = STRASSEN(A11+A12, B22)
    M6 = STRASSEN(A21-A11, B11+B12)
    M7 = STRASSEN(A12-A22, B21+B22)
    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6
    combine C11,C12,C21,C22 into result C
    return C

```

### ***Time / Space Complexity Analysis:***

Standard matrix multiplication performs  $n$  calls of an  $O(n)$  inner loop for each of the  $n^2$  output cells, giving  $O(n^3)$ . Strassen's algorithm reduces the number of recursive multiplications per split from 8 to 7 (at the cost of extra additions), giving a recurrence  $T(n) = 7T(n/2) + O(n^2)$ , which solves to  $O(n^{\log_2 7}) \approx O(n^{2.81})$ .

### ***Expected Observations & Graph Interpretation:***

For small matrices, the standard method is often faster in practice because Strassen's algorithm carries larger constant-factor overhead from the extra matrix additions and recursive call management. As matrix size grows, Strassen's asymptotic advantage becomes visible and its execution time increasingly outperforms the standard  $O(n^3)$  method.

### ***Conclusion & Justification:***

Strassen's method provides a genuine computational advantage only for sufficiently large matrices, where the reduced asymptotic exponent outweighs its constant-factor overhead; for small or moderate matrix sizes typical in many applications, the standard method remains simpler and competitively fast.

## **Experiment 5: Trade-offs Between Merge Sort and Quick Sort**

### ***Aim:***

To compare Merge Sort and Quick Sort in terms of execution time, space complexity, and stability, and to justify which algorithm is more suitable for large-scale applications under different data conditions.

### ***Algorithm Design & Approach:***

Both algorithms are implemented and executed on identical datasets of increasing sizes and varying distributions (random, sorted, reverse-sorted, duplicate-heavy). Execution time, additional memory allocated, and whether the relative order of equal-valued elements is preserved (stability) are recorded for each run.

### ***Pseudocode:***

```

MERGE_SORT(A, low, high)
    if low < high

```

```

    mid = (low + high) / 2
    MERGE_SORT(A, low, mid)
    MERGE_SORT(A, mid+1, high)
    MERGE(A, low, mid, high)

MERGE(A, low, mid, high)
    create temp arrays L[low..mid], R[mid+1..high]
    i = low ; j = mid+1 ; k = low
    while i <= mid and j <= high
        if A[i] <= A[j]: A_temp[k++] = A[i++]
        else:           A_temp[k++] = A[j++]
    copy remaining elements of L and R into A_temp
    copy A_temp back into A[low..high]

QUICK_SORT(A, low, high)
    if low < high
        p = PARTITION(A, low, high)    // pivot chosen per strategy
        QUICK_SORT(A, low, p-1)
        QUICK_SORT(A, p+1, high)

PARTITION(A, low, high)    // Lomuto scheme, pivot = A[high]
    pivot = A[high]
    i = low - 1
    for j = low to high-1
        if A[j] <= pivot
            i = i + 1
            swap A[i], A[j]
    swap A[i+1], A[high]
    return i + 1

// Pivot-selection variants (choose before calling PARTITION):
//   First pivot:  swap A[low], A[high]  before partitioning
//   Random pivot: r = random(low, high); swap A[r], A[high]

```

### ***Time / Space Complexity Analysis:***

Merge Sort:  $O(n \log n)$  time in all cases,  $O(n)$  auxiliary space, and is stable (equal elements retain relative order because the merge step takes from the left sub-array when values are equal). Quick Sort:  $O(n \log n)$  average time but  $O(n^2)$  worst case on adversarial pivots,  $O(\log n)$  auxiliary space (recursion stack) for in-place partitioning, and is not stable (partitioning can reorder equal elements).

### ***Expected Observations & Graph Interpretation:***

Quick Sort is typically faster in practice on random data due to lower constant factors and in-place partitioning (better cache locality), while Merge Sort shows more consistent timing across all data distributions and uses noticeably more memory due to the auxiliary merge arrays.

### ***Conclusion & Justification:***

Quick Sort is generally preferred for large-scale, memory-constrained, in-place sorting of random data where average-case speed matters most, while Merge Sort is preferred when guaranteed  $O(n \log n)$  performance, stability, or external/large-scale sorting (e.g. sorting data that doesn't fit in memory) is required.

## **Experiment 6: Recursive vs Iterative Binary Search**

### ***Aim:***

To analyze the performance of recursive and iterative Binary Search implementations by recording execution time and memory usage on datasets of varying sizes, and to justify which approach is more suitable in practical applications.

### ***Algorithm Design & Approach:***

Both recursive and iterative Binary Search are implemented with identical comparison logic, differing only in control structure. Each is executed on sorted datasets of increasing sizes, and both execution time and call-stack/memory usage are recorded.

### ***Pseudocode:***

```
ITERATIVE_BINARY_SEARCH(A, n, key)
    low = 0 ; high = n - 1
    while low <= high
        mid = low + (high - low) / 2
        if A[mid] == key: return mid
        else if A[mid] < key: low = mid + 1
        else: high = mid - 1
    return -1

RECURSIVE_BINARY_SEARCH(A, low, high, key)
    if low > high: return -1
    mid = low + (high - low) / 2
    if A[mid] == key: return mid
    else if A[mid] < key:
        return RECURSIVE_BINARY_SEARCH(A, mid+1, high, key)
    else:
        return RECURSIVE_BINARY_SEARCH(A, low, mid-1, key)
```

### ***Time / Space Complexity Analysis:***

Both approaches share the same  $O(\log n)$  time complexity in best, average, and worst cases, since each halves the search interval identically. However, the recursive version additionally uses  $O(\log n)$  auxiliary space for the call stack (one stack frame per recursive call), whereas the iterative version uses only  $O(1)$  auxiliary space.

### ***Expected Observations & Graph Interpretation:***

Execution time for both versions is expected to be very close for small and moderate  $n$ , but the recursive version shows measurably higher memory usage and slightly higher execution time overhead due to function-call bookkeeping (stack frame creation/teardown) at large  $n$  or in languages/environments with expensive call overhead.

### ***Conclusion & Justification:***

Recursion makes the algorithm's divide-and-conquer structure clearer and easier to reason about, but the iterative version is more practical for performance- and memory-sensitive applications (e.g. embedded systems, deeply nested searches) since it avoids stack-depth growth and function-call overhead entirely.

## **Experiment 7: Quick Sort Worst-Case Behaviour on Sorted and Reverse-Sorted Data**

### ***Aim:***

To evaluate Quick Sort's behaviour under worst-case conditions by executing it on already-sorted and reverse-sorted datasets, comparing results with random inputs, and justifying techniques to avoid worst-case scenarios.

### ***Algorithm Design & Approach:***

Quick Sort (using a fixed first/last-element pivot) is executed on sorted, reverse-sorted, and randomly shuffled datasets of the same size, and execution time is recorded for each. The experiment is then repeated using a randomized pivot strategy for comparison.

### ***Pseudocode:***

```
QUICK_SORT(A, low, high)
    if low < high
        p = PARTITION(A, low, high)    // pivot chosen per strategy
        QUICK_SORT(A, low, p-1)
        QUICK_SORT(A, p+1, high)

PARTITION(A, low, high)    // Lomuto scheme, pivot = A[high]
    pivot = A[high]
    i = low - 1
    for j = low to high-1
        if A[j] <= pivot
            i = i + 1
            swap A[i], A[j]
    swap A[i+1], A[high]
    return i + 1

// Pivot-selection variants (choose before calling PARTITION):
//   First pivot:  swap A[low], A[high]  before partitioning
//   Random pivot: r = random(low, high); swap A[r], A[high]
```

### ***Time / Space Complexity Analysis:***

With a fixed pivot on already-sorted or reverse-sorted input, every partition produces one empty sub-array and one sub-array of size  $n-1$ , giving the recurrence  $T(n) = T(n-1) + O(n)$ , which solves to  $O(n^2)$ . On random input, partitions are balanced on average, giving the expected  $O(n \log n)$ .

### ***Expected Observations & Graph Interpretation:***

Execution time on sorted/reverse-sorted input with a fixed pivot is expected to grow quadratically and be dramatically higher than on random input of the same size, while the randomized-pivot version shows execution time close to the random-input case for all three data conditions.

### ***Conclusion & Justification:***

Fixed pivot selection makes Quick Sort's worst case trivially triggerable by common real-world input patterns (already-sorted data); randomized pivot selection, or median-of-three selection, removes this predictable weakness and is therefore recommended for any general-purpose Quick Sort implementation.

## **Experiment 8: Scalability Analysis of Merge Sort for Large Datasets**

### ***Aim:***

To analyze the scalability of Merge Sort by applying it to datasets of increasing size ( $10^3$ ,  $10^4$ ,  $10^5$  elements), plotting the growth trend, and justifying its suitability for large-scale data processing.

### ***Algorithm Design & Approach:***

Random datasets of size  $10^3$ ,  $10^4$ , and  $10^5$  (and optionally  $10^6$ ) elements are generated, and Merge Sort's execution time is recorded for each size. The results are plotted with input size on the x-axis and execution time on the y-axis, and compared against the theoretical  $n \log n$  curve.

### ***Pseudocode:***

```
MERGE_SORT(A, low, high)
    if low < high
        mid = (low + high) / 2
        MERGE_SORT(A, low, mid)
        MERGE_SORT(A, mid+1, high)
        MERGE(A, low, mid, high)
```



```

MERGE(A, low, mid, high)
  create temp arrays L[low..mid], R[mid+1..high]
  i = low ; j = mid+1 ; k = low
  while i <= mid and j <= high
    if A[i] <= A[j]: A_temp[k++] = A[i++]
    else:           A_temp[k++] = A[j++]
  copy remaining elements of L and R into A_temp
  copy A_temp back into A[low..high]

```

### ***Time / Space Complexity Analysis:***

Merge Sort's time complexity is  $O(n \log n)$  for all input distributions, and its recursion depth is  $O(\log n)$ , which stays modest even for very large  $n$  (e.g.  $\log_2(10^6) \approx 20$ ), so stack usage remains bounded and predictable as input size grows.

### ***Expected Observations & Graph Interpretation:***

Plotting execution time divided by  $n$  (i.e. time/ $n$ ) against  $\log(n)$  is expected to produce an approximately linear/flat trend, confirming that the growth in total time closely tracks  $n \log n$  rather than a steeper polynomial curve — Merge Sort's execution time roughly increases by a constant factor of  $\log(10)$  each time  $n$  grows by  $10x$ , not by a factor of  $10$  as a linear algorithm's time would.

### ***Conclusion & Justification:***

The asymptotic  $O(n \log n)$  bound and low, predictable recursion depth make Merge Sort well-suited for large-scale data processing, including scenarios where sorting must be distributed, parallelized, or performed as external sorting on data that exceeds available memory.

## **Experiment 9: Efficiency of Divide-and-Conquer Matrix Multiplication vs Standard Iterative Approach**

### ***Aim:***

To evaluate the efficiency of divide-and-conquer matrix multiplication (recursive block multiplication, or Strassen's method) against the standard iterative approach, and to justify the advantages and limitations of divide-and-conquer in matrix operations.

### ***Algorithm Design & Approach:***

Square matrices of increasing sizes are multiplied using (a) the standard triple-nested-loop approach and (b) a recursive divide-and-conquer approach that splits each matrix into four quadrant sub-matrices and combines eight (or seven, for Strassen's optimization) recursive sub-multiplications. Execution time is recorded for both methods across matrix sizes.

### ***Pseudocode:***

```

STANDARD_MULTIPLY(A, B, n)    // n x n matrices
  for i = 0 to n-1
    for j = 0 to n-1
      C[i][j] = 0
      for k = 0 to n-1
        C[i][j] += A[i][k] * B[k][j]
  return C

STRASSEN_MULTIPLY(A, B, n)    // n x n, n a power of 2
  if n == 1: return [[ A[0][0] * B[0][0] ]]
  split A, B into four (n/2)x(n/2) submatrices:
    A11,A12,A21,A22 and B11,B12,B21,B22
  M1 = STRASSEN(A11+A22, B11+B22)

```

```

M2 = STRASSEN(A21+A22, B11)
M3 = STRASSEN(A11, B12-B22)
M4 = STRASSEN(A22, B21-A11)    // note: A22 * (B21 - B11)
M5 = STRASSEN(A11+A12, B22)
M6 = STRASSEN(A21-A11, B11+B12)
M7 = STRASSEN(A12-A22, B21+B22)
C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6
combine C11,C12,C21,C22 into result C
return C

```

### ***Time / Space Complexity Analysis:***

Standard multiplication:  $O(n^3)$ . Naive divide-and-conquer (8 recursive multiplications per split) also resolves to  $O(n^3)$ , offering no asymptotic gain by itself. Strassen's 7-multiplication reduction achieves  $O(n^{\log_2 7}) \approx O(n^{2.81})$ , a genuine asymptotic improvement, at the cost of extra matrix additions and higher implementation complexity.

### ***Expected Observations & Graph Interpretation:***

The naive recursive divide-and-conquer approach is expected to perform similarly to (or slightly worse than, due to recursive overhead) the standard method, while Strassen's approach shows a growing advantage only once matrix sizes become large enough for the reduced exponent to outweigh its constant-factor overhead.

### ***Conclusion & Justification:***

Divide-and-conquer alone (without reducing the number of recursive multiplications) does not improve on standard multiplication's complexity; the real benefit comes specifically from algorithms like Strassen's that reduce the branching factor of the recursion, and even then the benefit is only realized for sufficiently large matrices due to overhead and numerical stability concerns.

## **Experiment 10: Impact of Data Distribution on Sorting Performance (Merge Sort vs Quick Sort)**

### ***Aim:***

To analyze the impact of data distribution — random, nearly sorted, and duplicate-heavy inputs — on the performance of Merge Sort and Quick Sort, and to justify which algorithm performs better under each condition.

### ***Algorithm Design & Approach:***

Merge Sort and Quick Sort are each executed on three dataset types of equal size: randomly shuffled data, nearly sorted data (a small fraction of elements out of place), and duplicate-heavy data (many repeated values). Execution time is recorded for every algorithm/distribution combination.

### ***Pseudocode:***

```

MERGE_SORT(A, low, high)
    if low < high
        mid = (low + high) / 2
        MERGE_SORT(A, low, mid)
        MERGE_SORT(A, mid+1, high)
        MERGE(A, low, mid, high)

MERGE(A, low, mid, high)
    create temp arrays L[low..mid], R[mid+1..high]
    i = low ; j = mid+1 ; k = low
    while i <= mid and j <= high
        if A[i] <= A[j]: A_temp[k++] = A[i++]

```

```

    else:
        A_temp[k++] = A[j++]
    copy remaining elements of L and R into A_temp
    copy A_temp back into A[low..high]

QUICK_SORT(A, low, high)
    if low < high
        p = PARTITION(A, low, high)    // pivot chosen per strategy
        QUICK_SORT(A, low, p-1)
        QUICK_SORT(A, p+1, high)

PARTITION(A, low, high)    // Lomuto scheme, pivot = A[high]
    pivot = A[high]
    i = low - 1
    for j = low to high-1
        if A[j] <= pivot
            i = i + 1
            swap A[i], A[j]
    swap A[i+1], A[high]
    return i + 1

// Pivot-selection variants (choose before calling PARTITION):
//   First pivot:  swap A[low], A[high]  before partitioning
//   Random pivot: r = random(low, high); swap A[r], A[high]

```

### ***Time / Space Complexity Analysis:***

Merge Sort remains  $O(n \log n)$  regardless of distribution, since its splitting and merging costs do not depend on element values. Quick Sort (with fixed pivot) can degrade toward  $O(n^2)$  on nearly sorted data (predictable pivot placement) and can suffer from unbalanced partitions on duplicate-heavy data unless a three-way partitioning scheme is used, in which case duplicates are grouped in  $O(n)$  and Quick Sort's performance actually improves.

### ***Expected Observations & Graph Interpretation:***

Merge Sort's execution time is expected to stay essentially flat across all three distributions. Quick Sort (standard two-way partition) is expected to slow down noticeably on nearly sorted data with a fixed pivot, and may show increased or decreased performance on duplicate-heavy data depending on whether three-way partitioning is used.

### ***Conclusion & Justification:***

Merge Sort is the more predictable and distribution-independent choice, making it preferable when input characteristics are unknown or adversarial; Quick Sort can match or exceed Merge Sort's practical speed on random or duplicate-heavy data (with three-way partitioning) but requires randomized pivoting or hybrid techniques to remain safe on nearly sorted or adversarial inputs.